

Company: Amazon

Interview Type: DSA

Q1: Warehouse Robot: Shortest Path with Limited Power-Ups

Difficulty: Hard

Tags: BFS, State-Space Search, Bitmask, Grid

Problem Statement: You are given an $N \times M$ grid representing a warehouse. The grid contains obstacles (1), empty spaces (0), a starting point (S), an exit (E), and K power-up stations (P). A robot can move in four directions. Normally, it cannot pass through obstacles. However, it can collect a "Shield" power-up from a station (P) that allows it to pass through exactly one obstacle. Each shield is single-use, but the robot can carry up to K shields simultaneously. Find the minimum number of steps to reach the exit (E).

Expected Approach: Use Breadth-First Search (BFS) to find the shortest path.

Optimized Approach: Standard BFS only tracks coordinates (x, y) . To optimize, use a state-based BFS with the state $(x, y, \text{shields_collected})$. The shields_collected can be tracked as an integer count. Maintain a 3D visited array $\text{visited}[N][M][K+1]$ to prevent redundant processing of the same location with the same or fewer shields.

Time Complexity: $O(N * M * K)$

Space Complexity: $O(N * M * K)$

Optimizations: Use a bitmask if power-up stations are unique and need to be visited only once, but since they are interchangeable here, a simple count is sufficient.

Hints: Think of this as a layered graph where each layer represents the number of shields currently held.

Q2: Amazon Fresh: Maximum Profit with Delivery Dependencies

Difficulty: Hard

Tags: Greedy, Priority Queue, Disjoint Set Union (DSU)

Problem Statement: You have N delivery jobs. Each job i has a profit $p[i]$ and a deadline $d[i]$. Each job takes exactly one unit of time to complete. However, some jobs have a "dependency" constraint: job A must be completed before job B . If you complete a job after its deadline, you receive zero profit. Determine the maximum profit you can achieve.

Expected Approach: Sort jobs by profit in descending order and try to schedule each job as late as possible before its deadline.

Optimized Approach: Use a Max-Heap to store available profits and a Disjoint Set Union (DSU) to efficiently find the latest available time slot for a job. For dependencies, perform a Topological Sort first to establish a valid processing order. Combine these by iterating through the topologically sorted jobs

and using a DSU-based greedy approach to assign the best available time slot $t \leq \text{deadline}$.

Time Complexity: $O(N \log N + N \alpha(T))$, where T is the maximum deadline.

Space Complexity: $O(N + T)$

Optimizations: Use DSU to avoid $O(T)$ linear scans when searching for free time slots; the "parent" of a slot t points to the next available slot $t-1$.

Hints: To maximize profit, always try to pick the most profitable job available, but ensure its dependencies and deadlines are respected using a greedy strategy.

Q3: Optimal Fulfillment Assignment

Difficulty: Medium-Hard

Tags: Greedy, Sorting, Two Pointers

Problem Statement: Amazon has n delivery robots with skill levels represented by an array `skill` and m delivery tasks with difficulty levels represented by an array `tasks`. You are given k "performance boosters." Each booster can increase a single robot's skill level by exactly 1. A robot can complete a task if its skill level (after boosters) is greater than or equal to the task's difficulty level. Each robot can be assigned at most one task. Find the maximum number of tasks that can be completed.

Expected Approach: Sort both arrays. Use a greedy approach with a two-pointer technique. For each task (starting from the easiest), try to assign the weakest robot that can complete it without boosters. If no such robot exists, use the minimum required boosters on the weakest available robot that can reach the required difficulty.

Optimized Approach: Sort `skill` and `tasks` in ascending order. Iterate through tasks using a pointer `j`. For each task, find the first robot `i` that can satisfy `skill[i] >= tasks[j]`. If `skill[i] < tasks[j]`, check if `(tasks[j] - skill[i]) <= k`. If yes, subtract the needed boosters from `k` and move to the next task/robot. If not, this robot can never fulfill this task or any harder task; move to the next robot.

Time Complexity: $O(N \log N + M \log M)$ due to sorting.

Space Complexity: $O(1)$ or $O(N)$ depending on the sorting implementation.

Optimizations: Use binary search to find the first robot that can potentially complete a task to skip iterations, though Two Pointers is generally more efficient for the full set.

Hints: Should you assign your strongest robot to the hardest task or the easiest task? Think about the conservation of boosters.

Q4: Customer Sentiment Stability

Difficulty: Hard

Tags: Sliding Window, Monotonic Queue, Deque

Problem Statement: Given an array of integers `scores` representing the sentiment of product reviews over time, find the total number of contiguous subsegments (subarrays) where the "instability" is less than or equal to `X`. Instability is defined as the absolute difference between the maximum and

minimum sentiment score within that subsegment.

Expected Approach: A nested loop approach $O(N^2)$ to check every possible subarray and find its min/max.

Optimized Approach: Use a sliding window with two monotonic deques. Maintain one deque to track the maximum value in the current window and another to track the minimum value. For each right pointer `r`, update the deques. While $\text{max_deque.front()} - \text{min_deque.front()} > X$, increment the left pointer `l` and remove indices from the deques that are out of bounds. The number of valid subarrays ending at `r` is $r - l + 1$. Sum these values for the result.

Time Complexity: $O(N)$ as each element is added and removed from the deques at most once.

Space Complexity: $O(N)$ to store the deques in the worst case.

Optimizations: Use primitive arrays instead of Deque objects in high-performance environments to reduce overhead.

Hints: If a subarray $[l, r]$ is stable, what can you say about the subarray $[l+1, r]$? How can you track the min and max of a sliding window in constant time?

Q5: Logistics Path with Energy Resets

Difficulty: Hard

Tags: Binary Search, Dijkstra, Grid

Problem Statement: An automated robot navigates an N times M grid to deliver packages. Each cell (r, c) has an elevation $H_{r,c}$. Moving from a cell to an adjacent cell (nr, nc) consumes battery: if $H_{nr,nc} > H_{r,c}$, the cost is $(H_{nr,nc} - H_{r,c})^2$; otherwise, the cost is 1 unit. Specific cells are "Solar Pads" that instantly recharge the robot to its maximum capacity CC . Given a start SS and a destination DD , find the minimum capacity CC required to guarantee the robot can reach DD without the battery level ever dropping below zero.

Expected Approach: Use binary search on the possible capacity CC . For a fixed CC , use BFS or Dijkstra to determine if DD is reachable.

Optimized Approach: Use a modified Dijkstra algorithm. Maintain a 2D array $\text{max_rem}[r][c]$ representing the maximum remaining battery charge when reaching cell (r, c) . In the priority queue, store $(\text{remaining_battery}, r, c)$ and always expand the node with the highest remaining battery. If a cell is a Solar Pad, the remaining battery resets to CC . This allows checking a fixed CC in $O(NM \log(NM))$. Combined with binary search on CC , the result is found.

Time Complexity: $O(N \cdot M \log(N \cdot M) \log(\text{Max_Elevation}^2))$

Space Complexity: $O(N \cdot M)$

Optimizations: Use a 1D array for the priority queue keys and a simple BFS if all edge weights were uniform, but Dijkstra is necessary here due to the non-uniform battery costs and resets.

Hints: If you knew the capacity was CC , how would you check reachability? Does a higher capacity always make the destination more reachable?

Q6: Fulfillment Network Load Balancing

Difficulty: Hard

Tags: Tree, DFS, Greedy

Problem Statement: Amazon's fulfillment network is structured as a tree of N warehouses connected by $N-1$ bidirectional roads. Each warehouse i currently holds A_i units of a specific product. The total number of units $\sum A_i$ is guaranteed to be equal to $N \times K$, where K is the target number of units each warehouse should hold for optimal distribution. Moving one unit of the product between two adjacent warehouses costs 1. Find the minimum total cost to redistribute the units so that every warehouse ends up with exactly K units.

Expected Approach: Use a flow-based approach or multiple passes to move units from "surplus" nodes to "deficit" nodes.

Optimized Approach: Perform a single DFS traversal. For any subtree rooted at node u , calculate its "net balance": $\text{Balance}(u) = (\sum_{v \in \text{subtree}(u)} A_v) - (\text{count of nodes in subtree}(u) \times K)$. This balance represents the number of units that *must* be moved across the edge connecting u to its parent to achieve equilibrium. The total minimum cost is the sum of the absolute values of the balances of all nodes except the root.

Time Complexity: $O(N)$

Space Complexity: $O(N)$ for the adjacency list and recursion stack.

Optimizations: The balance can be calculated in a single post-order traversal where each node returns the sum of its own units and the units of its children to the parent.

Hints: Think about a leaf node. If it has a surplus, where must that surplus go? If it has a deficit, where must it come from? Does the specific path of a single unit matter, or just the net flow across each edge?

Q7: Minimum Energy Path with Charging Stations

Difficulty: Hard

Tags: Graph, Dijkstra, State-Space BFS

Problem Statement: A warehouse robot needs to travel from a starting bin S to a target bin E on a grid. The robot has a maximum battery capacity C (units of energy). Moving to an adjacent cell consumes 1 unit of energy. Some cells in the grid are "Charging Stations" that instantly refill the battery to C . The grid contains obstacles that the robot cannot pass through. Find the minimum number of steps to reach the target. If the robot's battery reaches 0 before reaching the target or a charging station, it can no longer move.

Expected Approach: Use a Breadth-First Search (BFS) to find the shortest path, but the state must track more than just coordinates.

Optimized Approach: Use a 3D state BFS $(x, y, \text{current_battery})$. For each cell, maintain the maximum battery level reached at that position to prune redundant paths. If a cell is a charging station, the current_battery in the next state resets to C . Use a $\text{visited}[x][y][\text{battery}]$ array or a hash map to keep track of visited states.

Time Complexity: $O(N * M * C)$, where N and M are grid dimensions and C is battery capacity.

Space Complexity: $O(N * M * C)$ to store the visited states.

Optimizations: Use Dijkstra's algorithm if edges have different weights (e.g., different terrain types). Use A* search with Manhattan distance as a heuristic to improve search speed.

Hints: Think about why a standard 2D BFS is insufficient. How does the battery level affect whether a future move is possible?

Q8: Maximum Total Value of Non-Overlapping Shipment Batches

Difficulty: Medium-Hard

Tags: Dynamic Programming, Binary Search, Sorting

Problem Statement: You are given an array of shipment requests, where each request i has a `start_time`, `end_time`, and a `profit`. Due to limited loading dock capacity, you can only process one shipment at a time. However, there is a special "overlap" rule: you can process a second shipment if its `start_time` is exactly equal to the `end_time` of the previous shipment. Find the maximum total profit you can achieve by selecting a subset of non-overlapping shipments.

Expected Approach: Sort shipments by `end_time`. Use dynamic programming where `dp[i]` represents the maximum profit using a subset of the first i shipments.

Optimized Approach: For each shipment i , use binary search (specifically `upper_bound` on `end_times`) to find the latest shipment j that ends before or at `shipment[i].start_time`. The transition is `dp[i] = max(dp[i-1], shipment[i].profit + dp[j])`.

Time Complexity: $O(N \log N)$ due to sorting and binary search for each shipment.

Space Complexity: $O(N)$ to store the DP table and sorted shipments.

Optimizations: If times are in a small discrete range, a Fenwick tree or Segment Tree could update values, but sorting + binary search is generally more robust for large time ranges.

Hints: This is a variation of the Weighted Interval Scheduling problem. How does the "start == end" rule affect your search for the previous compatible shipment?

Q9: Optimal Batch Partitioning for Warehouse Robots

Difficulty: Hard

Tags: Binary Search, Monotonic Queue, Greedy, Array

Problem Statement: Amazon warehouse robots process a sequence of N packages, each with a specific weight w_i . You need to partition this sequence into exactly K contiguous batches. For each batch, the "load variance" is defined as the difference between the maximum weight and the minimum weight in that batch. Design an algorithm to minimize the maximum load variance among all K batches.

Expected Approach: Use Dynamic Programming where $DP[i][j]$ represents the minimum maximum variance for the first i packages split into j batches. This results in $O(N^2 K)$ complexity, which is too slow for large N .

Optimized Approach: Binary Search on the Answer. The possible variance ranges from 0 to $\max(weights) - \min(weights)$. For a chosen variance X , use a Greedy approach with a

Monotonic Queue (or two pointers) to determine the minimum number of batches required such that no batch exceeds variance X . If the number of batches is $\leq K$, then X is feasible.

Time Complexity: $O(N \log(\text{Range of Weights}))$

Space Complexity: $O(1)$ (or $O(N)$ if using a deque for the sliding window check, though $O(1)$ suffices for a simple greedy pass in this specific variance constraint).

Optimizations: Pre-calculating min/max within the greedy check to avoid repeated $O(N)$ scans within the binary search loop.

Hints: 1. If we can solve the problem for a fixed variance X , can we find the optimal X ? 2. Is the feasibility function monotonic? 3. Think about how to greedily form a batch: keep adding elements until the current batch's $\max - \min > X$.

Q10: AWS Backbone Latency Optimization

Difficulty: Hard

Tags: Tree, DFS, Greedy, Binary Search

Problem Statement: An AWS region consists of N data centers connected in a tree topology ($N-1$ bidirectional links). You are tasked with placing M "Global Accelerators" in M different data centers. The goal is to minimize the maximum distance (number of hops) from any data center to its nearest Accelerator.

Expected Approach: BFS from all possible combinations of M nodes to find the minimum max distance. This is computationally expensive ($O(\binom{N}{M})$).

Optimized Approach: Binary Search on the possible distance D . For a fixed D , use a Greedy strategy on the tree to find the minimum number of Accelerators needed. Use a post-order traversal (DFS); for each node, track the distance to the furthest uncovered leaf in its subtree and the distance to the nearest Accelerator. If a leaf's distance to the nearest potential Accelerator placement reaches D , place an Accelerator at the highest possible ancestor to cover the most nodes.

Time Complexity: $O(N \log N)$

Space Complexity: $O(N)$ for the tree representation and recursion stack.

Optimizations: Use a greedy placement at the D -th ancestor of an uncovered node to maximize coverage per Accelerator.

Hints: 1. This is a variation of the "K-Center" problem, but on a tree. 2. A node can be covered by an Accelerator in its own subtree or by one in a different branch via an ancestor. 3. Root the tree arbitrarily and process from leaves up.
